Building vmlinuz and initrd.img

Building a custom Linux virtual machine (VM) image involves choosing an installation source or base image, configuring hardware and storage, installing the OS, and packaging the disk for deployment.

1. Choose a Method & Tool

- **Traditional ISO Installation:** Download a Linux distribution ISO (such as Ubuntu Server or Kali Linux) and install it interactively inside a hypervisor like Oracle VM VirtualBox, VMware Workstation Pro, Proxmox VE, or Hyper-V. [1, 2, 3, 4]
- **Cloud / Pre-built Base Images:** Download official cloud-init-enabled qcow2, vhd, or raw images directly from distribution cloud portals (e.g., Ubuntu Cloud Images) to boot instantly without a manual installer.
- **Container-to-VM (Bootc):** Define your Linux system inside a containerfile, build it via Podman/Docker, and convert the container image into a bootable virtual disk image. [1]

2. Configure Virtual Hardware

- **CPU & RAM:** Allocate processor cores (e.g., 2+ cores) and memory matching your workload (e.g., 2048 MB to 4096 MB+ for modern server/desktop environments). [1, 2]
- **Storage (Virtual Disk):** Create a virtual hard disk (VHD, VDI, QCOW2, or VMDK) with sufficient capacity (e.g., 20 GB to 60 GB+) and choose dynamic or fixed allocation. [1, 2]
- **Network:** Set the network adapter mode (NAT for isolated outbound traffic or Bridged for direct network visibility on your local LAN). [1, 2]
-

3. Install and Provision the OS

- Boot the VM from your ISO or disk image source and step through the installer (language, keyboard layout, partitioning, and user credentials).
- Install essential guest utilities for better integration (such as VirtualBox Guest Additions or Hyper-V integration services).
- Configure `cloud-init` if you plan to reuse the image across automated cloud or local deployments. [1, 2]

4. Generalize or Convert to a Template

- Clean up temporary files, machine-specific IDs (like `machine-id`), and log files.
- Shut down the VM and convert it into a reusable gold image or template (e.g., using Proxmox's **Convert to Template** feature or exporting the virtual disk file for distribution). [1]

Building an initial RAM disk (`initrd` or `initramfs`) for a Linux virtual machine (VM) involves creating a temporary root file system that the kernel loads into memory during the boot process to mount the actual root disk. [1, 2]

Key Tools for Building/Rebuilding an `initrd`

Depending on your Linux distribution, choose the appropriate tool:

- **Initramfs-tools** (`update-initramfs`, `mkinitramfs`): Standard on Debian, Ubuntu, and derivative distros.
- **Mkinitrd** (`mkinitrd`): Legacy or specific custom enterprise configurations.
- **CPIO + Gzip**: Manual construction method using archives for custom micro-kernels or minimal VMs. [1, 2, 3, 4, 5]

Common Scenarios & Steps

1. Rebuilding for Hypervisors (Adding Missing Drivers)

When moving a Linux image to a hypervisor like KVM, QEMU, or Hyper-V, you may need to inject specific storage or network bus drivers into the `initrd` so the kernel can see the virtual hard drive. [1]

- **Red Hat / CentOS / Fedora (using `mkinitrd` or `dracut`):**
  bash
  ```
  cd /boot
  sudo cp initrd-$(uname -r).img initrd-$(uname -r).img.bak
  sudo mkinitrd --preload=hv_storvsc --preload=hv_vmbus -v -f
  initrd-$(uname -r).img $(uname -r)
  ```
  Use code with caution.
  (Or via `dracut`):
  bash
  ```
  sudo dracut -f --add-driver "hv_vmbus hv_storvsc"
  /boot/initramfs-$(uname -r).img $(uname -r)
  ```
  Use code with caution.
- **Debian / Ubuntu (using `update-initramfs`):**
  Make sure your module names (e.g., `virtio_blk`, `virtio_net`) are listed in `/etc/initramfs-tools/modules`, then update:
  bash
  ```
  sudo update-initramfs -u -k all
  ```
  Use code with caution.

2. Manual Inspection and Disassembly

To inspect, modify, or inject custom scripts into an existing `initrd` or `initramfs` image: [1]

1. **Extract the archive:**
   mkdir custom_init && cd custom_init
   ```
   zcat /boot/initrd.img-$(uname -r) | cpio -idmv
   ```
   Use code with caution.
2. **Edit contents** (add custom binaries, config files, or modify the `/init` script).
3. **Rebuild the archive:**
   bash
   ```
   find . -print0 | cpio --null -o -H newc | gzip -c >
   /boot/new_initrd.img
   ```
   Use code with caution.

3. Testing with QEMU

Once your kernel (`bzImage` or `vmlinuz`) and `initrd` / `initramfs` are built, test them directly in QEMU before writing them to a full disk image: [1, 2]
bash
```
qemu-system-x86_64 -kernel bzImage -initrd initramfs.cpio.gz -append "console=ttyS0" -nographic
```
To build a custom Linux VM image containing `vmlinuz` (the compressed kernel) and `initrd`/`initramfs` (the initial RAM filesystem), you need to compile the kernel, build a minimal root/init structure, and package them together for a hypervisor like QEMU or KVM. [1, 2, 3, 4]

Step 1: Build the Linux Kernel (`vmlinuz`)

1. **Download source & configure:**
   bash
   ```
   tar -xvf linux-x.y.z.tar.xz
   cd linux-x.y.z
   make defconfig
   make menuconfig  # Customize drivers/options if needed
   ```
   Use code with caution.
2. **Compile the kernel:**
   bash
   ```
   make -j$(nproc)
   ```
   Use code with caution.
   - The compressed kernel image will be created at `arch/x86/boot/bzImage` (copy this to `/boot/vmlinuz`). [1, 2]
3. **Install modules:**
   bash
   ```
   make modules_install INSTALL_MOD_PATH=/path/to/rootfs
   ```
   Use code with caution.

Step 2: Build the Initial RAM Filesystem (`initrd`)

The `initrd`/`initramfs` contains a minimal directory tree and an executable `init` script that mounts the real root filesystem. [1, 2]

1. **Create directory structure:**
   bash
   ```
   mkdir -p initramfs/{bin,sbin,dev,proc,sys,etc,lib}
   ```
   Use code with caution.
2. **Add a minimal busybox or custom binaries:**
   - Compile or copy a static `busybox` binary into `initramfs/bin/` and create symlinks (`sh`, `mount`, `ls`, etc.).
3. **Write the `/init` script (`initramfs/init`):**
   bash
   ```
   #!/bin/sh
   mount -t proc none /proc
   mount -t sysfs none /sys
   mount -t devtmpfs none /dev
   ```

```
echo "Welcome to custom Linux VM!"
exec /bin/sh
```
Use code with caution.

- Make it executable: `chmod +x initramfs/init`

4. **Compress into `initrd.img` (or `initramfs.cpio.gz`):**
   bash
```
cd initramfs
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ../initrd.img
```
   Use code with caution.

Step 3: Boot the VM with QEMU

Test your newly built `vmlinuz` and `initrd.img` using QEMU:
bash
```
qemu-system-x86_64 \
  -kernel /boot/vmlinuz \
  -initrd /boot/initrd.img \
  -append "console=ttyS0 quiet" \
  -nographic \
  -m 512M
```

How to build the Linux kernel

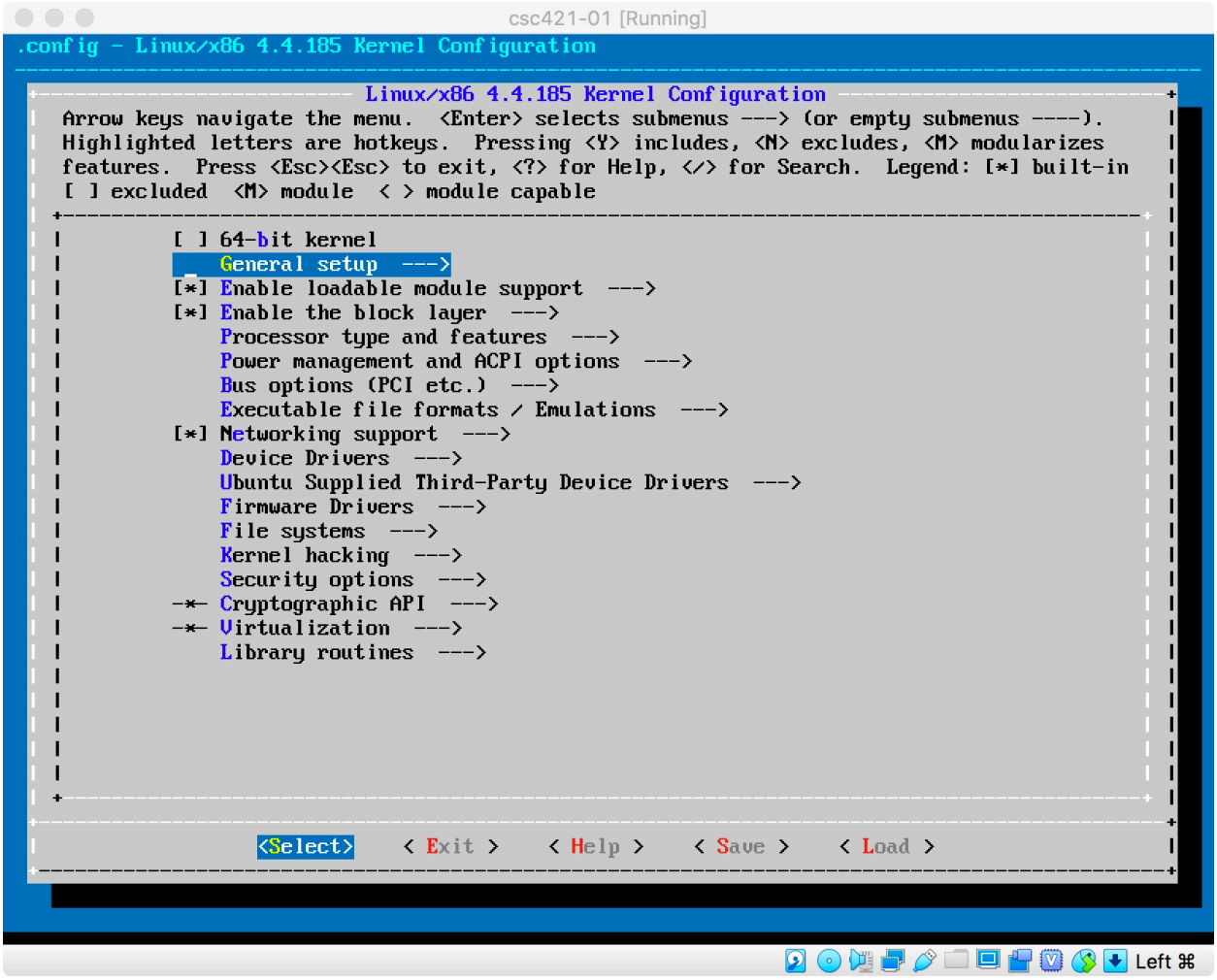
1. Create an Ubuntu 16.04 image in Virtual Box.
2. Login and become root (this helps with the build), using sudo -s
3. Use apt-get to install several software packages.
4. Use apt-get to install the linux source.
5. Unpack the linux source.

```
sudo -s
apt-get install build-essential ncurses-dev libssl-dev
apt-get install linux-source
apt-get --no-install-recommends install kernel-package
 cd /usr/src/linux-source-4.4.0
 tar jvxf linux-source-4.4.0.tar.bz2
```

12. In the proper linux-source-4.4.0 directory copy from /boot the current configuration into a template build configuration.
13. Run make menuconfig to customize and install the build configuration.

```
 cd /usr/src/linux-source-4.4.0/linux-source-4.4.0
 cp /boot/config-`uname -r` .config
 make menuconfig
```

Figure: The Makemenu panel. Use arrows, tabs, and return keys.



17. make menuconfig brings up an interactive set of panels. Use the arrow keys, the return key, and the tab key to navigate.
18. Modify the configuration so that your svn username is compiled right in. Suppose your name is pikachu, then,
    - arrow key to General setup; return
    - arrow key to Local version; return
    - type "-pika" into the empty box; return
    - tab key to "exit" on the bottom row of actions; return
    - tab key to "exit" on the bottom row of actions; return (yes, do this again)
    - return to accept "yes"
19. Make and install the kernel (you are still in the linux-source/linux-source directory).

```
20.                 make
21.                 make modules_install
22.                 make install
```

23. On a 2017 MacBook Pro with a 2.3 GHz Intel Core i5, 16GB RAM, the build takes almost two hours.
24. Check that the -csc421 versions of lib-modules, vmlinuz and initrd all exist (as the above screen shot).
25. Reboot and select in the Grub menu to boot either the new or old kernel.

A typical build will have output like this.

Extra notes:

For the 221 edition of the course, students now have M1 chips and some things have changed even for the 32-bit i386 architecture.

- For i686 aka i386 32 bit Intel,
  - apt-get install bc
- For aarch64 such as M1 Macs,
  - apt-get install bison flex
  - need to edit .config to change,

    ```
    CONFIG_SYSTEM_TRUSTED_KEYS="debian/canonical-certs.pem"
    ```

    to

    ```
    CONFIG_SYSTEM_TRUSTED_KEYS=""
    ```

  - unselect in menuconfig the NVIDIA Nouveau drivers
  - unselect in menuconfig the mellanox drivers (under device drivers > network device supprt > ethernet driver support)
  - unselect in menuconfig the staging drivers
  - To be continued, I am sure.

Grub boot screen

The Grub Boot Screen

A single linux install can boot to any of several kernels. You will not be overwriting the old kernel with the new kernel. You will be provided an option at boot to boot any of the installed kernels. Since the modules are in directories with names that correspond to the kernel names, the proper modules will be used on any boot.

The BIOS (firmware) know how to find a small boot code on a disk. The purpose of this code is to find the Boot Loader on the disk, to load it into memory and transfer control to it. The Boot Loader then finds the operating system kernel on the disk, loads it, and transfer control to it. The operating system does startup tasks, first in single user mode, then in multi-processing mode, before starting the init process, which makes the computer available for logins.

Historically, there have been three linux boot loaders: lilo, grub and grub2. Most Linux distros are using grub2. A step in the makefile that installs the built kernel is to call an update script that includes all known kernels found in /boot into the grub boot menu.

The images show my grub menu after following these instructions for a kernel build. Once the first screen appears, use any key to stop the automatic boot to the default kernel. Navigate to advanced options and enter, then navigate to the kernel of choice and enter.

You can modify grub2's boot behavior. If your grub is doing an automatic, silent boot to the default kernel, without the option to select a kernel, edit /etc/default/grub as follows:

```
#GRUB_HIDDEN_TIMEOUT=0 # comment out this line
GRUB_CMDLINE_LINUX_DEFAULT="" # set this to the empty string
```

then run update-grub. this will give a verbose boot with a pause to select among the alternative kernels.

Now you are ready to try your new kernel. Type reboot and your machine will reboot. At the boot menu, select your kernel. Hopefully it will boot properly. Once booted, use uname -r to see the revision number of the kernel you are running. If it is yours, congratulations!

The kernel configuration, in detail

Before building a kernel, you need to configure the build. This states the options that the kernel will include, will choose whether certain functionality is built-in or placed in a separable module; and what functionality and drivers will be omitted from the build altogether.

The master file for configuration is the hidden file .config which is placed in the root of the build tree. If unpacked your source into the directory /usr/src/linux-source-xyz, so the full pathname of the config file is /usr/src/linux-source-xyz/.config. To install the configuration, you must run a script, for example make oldconfig, to integrate into the Kconfig files that are scattered around the build tree those options specified in .config. (I think, I really don't understand this thoroughly.)

Other scripts are make defconfig, which creates a configuration based on the current platform, or make menuconfig that presents a curses (a ^5.2 Rogue) interface to make build selections. You will need ncurses-dev installed for menuconfig to work.

Make install does several things:

1. copies into /boot the built kernel as vmlinuz-[kernel revision name],
2. copies into /boot the .config file as config-[kernel revision name],
3. copies into /boot the symbol table as System.map-[kernel revision name],

4. creates in /boot an *initial root directory* named initrd.img-[kernel revision name],
5. copies all built the modules to /lib/modules/[kernel revision name].
6. updates the grub configuration for any changes to the grub boot menu.

Run make help in the root of the build source for information on the build targets. Modules are each a separate file, with a .ko extension. They are installed by being copied into the special directory /lib/modules/[kernel revision name]. Kernel modules are parts of the kernel that are dynamically loaded as needed.

RAM Disks, such as initrd

A *RAM disk* is a disk-like storage system that uses RAM as the final destination for the data rather than a magnetic disk. It is preferred for speed, however it is volatile — it goes away on reboot. Sometimes that's a plus — for example temporary files, that must to be erased on reboot anyway.

Linux uses a very intelligent *buffer cache* to shuttle data blocks to and from the disk. Each buffer in the buffer cache remembers what disk block it mirrors. A disk read can satisfied without going out to the disk if the block is sitting in the buffer cache, because the block has been recently read or written.

Given that the buffer cache exists, it was realized that a RAM disk can be created more efficiently by stopping the transfer between file system and disk at the buffer cache, and marking the buffer in the cache which carries data as "permanent".

This eliminates an essentially useless copy from one in-RAM buffer to another — from the buffer cache to the "RAM disk". Just sitting permanently in the buffer cache is sufficient to create an in-RAM file system.

This scheme was christened the *ramfs*.

After building and installing the kernel and the modules, you need to build the *initial root directory*. In case you google for additional help on this, beware that the initial root directory process has changed recently. It now uses a simple cpio into a ramfs, rather than imaging a RAM-disk. (See sidebar for discussion of Ramfs versus RAM-disk.)

At boot, the boot loader knows only enough about reading from disk so that it can copy the kernel from a file on disk into the memory, and then begin running the kernel (early boot). The kernel itself might rely on modules to provide the code necessary to access the file systems on disk. These modules are files that might reside on the file systems which the kernel cannot yet read.

Linux solves this problem by having a partition on disk in a file format that the boot loader and the initial kernel know how to read. The kernel is stored in this partition, as well as an archive of the initial file system required by the kernel during early

boot. The kernel creates a ramfs and unpacks this archive into the ramfs and mounts this as the initial root directory.

This partition is typically ext2, and is mounted into the fully booted file system at the name /boot.

This root serves the kernel during early boot, but is later jettisoned when the real root directory is discovered on disk and mounted over the initial root, called *re-rooting* or *pivoting on root*.

The process of creating a initial root directory is automated by the update-initramfs script. This is generally called as part of the default build, and you can also see it run during certain kernel updates. If this file is not correct, the boot process will go well up until the first file access to a file system. The error messages will be about not finding a root file system, or not recognizing it. That means the initrd was not properly formed or placed.

Understanding the initrd and vmlinuz in Linux Boot Process

A Technical Analysis of Core Linux Boot Components

In this article, we will conduct a detailed examination of two critical elements in the Linux boot process: **vmlinuz** and **initrd**, essential for any systems engineer or developer working with Linux.

vmlinuz: The Compressed Linux Kernel

vmlinuz is the compressed, bootable Linux kernel image. The name is derived from "Virtual Memory LINUx gZip," indicating that it is a gzip-compressed Linux kernel. This compressed format is crucial for reducing boot time and memory footprint.

The structure of vmlinuz includes a preliminary setup routine at the image's head. This routine is responsible for minimal hardware initialization, following which it decompresses the kernel image into high memory. The decompression is performed in-place, and the decompressed kernel is executed directly from the high memory, transitioning control to the kernel's main function.

initrd: Initial RAM Disk

initrd stands for "initial RAM disk," a temporary root filesystem used in the boot process. The initrd is loaded by the bootloader along with the kernel image and is essential for the two-stage boot process that modern Linux systems use.

The initrd serves as an interim root filesystem until the actual root filesystem is mounted. It contains a minimal set of directories and executables, primarily for kernel module loading. Tools like `insmod` are included in initrd to facilitate this.

The primary function of initrd is to make the real filesystems available, whether they are on local storage or network resources. This is particularly important for systems that require kernel modules to access the disk controllers or filesystems of the root partition.

The Boot Process Involving vmlinuz and initrd

1. **Loading Stage**: The bootloader (like GRUB) loads both the vmlinuz and initrd into memory.
2. **Decompression of vmlinuz**: The embedded routine in vmlinuz decompresses the kernel into memory.

3. **Handoff to initrd**: Post-decompression, control is passed to the kernel, which then mounts the initrd as its initial root filesystem.
4. **Module Loading**: The initrd's primary role is to load necessary modules. These modules are crucial for the kernel to access the hardware required to mount the real root filesystem.
5. **Transition to Actual Root Filesystem**: Once the necessary drivers are loaded, the kernel can mount the real root filesystem and continue the boot process.

In summary, vmlinuz and initrd are integral to the Linux boot process. vmlinuz, as a compressed kernel image, reduces the initial load time and memory usage, while initrd provides a temporary root filesystem to facilitate the kernel in loading necessary modules to access the actual root filesystem. This design allows Linux to maintain a balance between a fast boot time and the flexibility to support a wide range of hardware configurations. Understanding these components is crucial for anyone involved in Linux kernel development, system administration, or related fields.

https://kpj.github.io/linux/KernelBuilding.html